

SAP Cloud Integration: Dynamic Stateless Architecture Blueprint

Target State: Zero Persistent Storage, Zero Database, On-Demand Server RAM Pass-Through with Trace Scraping Loophole

1. Executive Summary & Design Constraints

This architecture blueprint specifies the technical implementation of an interactive operations dashboard layered on top of **SAP Cloud Integration (CI)**. To satisfy strict corporate data residency rules, information privacy regulations, and resource limits, the system operates under a hard technical ban on the following components:

- **No Storage Footprint:** No external relational database, Document DB, or key-value disk store.
- **No Native Persistence:** No internal SAP CI JMS Queues or Data Store structures are permitted.
- **No Code Modifications:** No edits or Exception Subprocess updates to existing production integration flows.

Compliance is achieved using the **On-Demand Transient Runtime Scraper** pattern. The application treats message payloads and heavy binary attachments as ephemeral data streams, buffer-mapping them exclusively within volatile **Server RAM (Random Access Memory)** for microsecond durations during active re-delivery cycles before systematic erasure.

2. Operational Data Lifecycle Lifecycle Matrix

Phase	Trigger Event	Data State & Storage Medium	Footprint
1. Log Discovery	User authentication / session load.	Metadata only (GUID, Flow ID, Error String) fetched via OData API.	0 Bytes (UI Text rendering only)
2. On-Demand Fetch	User clicks the 'Resend & Inspect' button.	Dashboard Backend pulls payload and attachment binary streams from SAP Trace API.	Temporary allocation in Volatile Server RAM
3. Runtime Proxy	Automatic follow-up trigger loop.	Backend pipes Multipart Form streams straight to standard HTTP runtime endpoints.	Active memory transit
4. Garbage Collection	HTTP Receiver confirmation (200/201 OK).	JavaScript variable reference is broken. V8 engine de-allocates blocks immediately.	0 Bytes (Permanently purged)

3. Technical Prerequisites & SAP CI Configuration

Because the platform is stateless, the metadata and payload must be scraped directly from standard infrastructure logs. This requires a specific runtime state on the SAP tenant:

1. **Log Level = Trace:** The log level of targeted integration flows must be actively switched to *Trace* within the SAP Monitoring cockpit. This tells SAP to hold execution snapshots (payloads/attachments) in its core engine for **exactly 10 minutes** before an automatic infrastructure purge occurs.
2. **Process Integration Runtime API Key:** An instance configured with the **api** plan must hold the explicit MonitoringDataRead and AuthGroup_Administrator parameters inside its security roles array to facilitate the background scraping loop.

4. Node.js Stateless Engine Implementation

The backend acts as a stateless proxy pipe. It queries the SAP OData APIs, fetches trace binaries on-the-fly, routes them back to the worker node, and drops variable references immediately to trigger memory release.

```
// Production-Grade Dynamic Resend & Attachment Scraper Pipeline
const express = require('express');
const axios = require('axios');
const app = express();
app.use(express.json());

const CPI_API_URL = "https://your-tenant-tmm.itc.cfapps.eu10.hana.ondemand.com";

app.post('/api/dashboard/dynamic-resend/:msgGuid', async (req, res) => {
  const { msgGuid } = req.params;
  const { targetIflowHttpAddress } = req.body; // e.g., '/http/ERP_to_CRM'

  try {
    const token = req.headers.authorization; // Relays the dynamic BTP OAuth bearer token

    // Step 1: Scrape Attachment Manifest from SAP Trace API
    const manifestUrl = `${CPI_API_URL}/api/v1/MessageProcessingLogs('${msgGuid}')/MessageProcess
ingLogAttachments`;
    const manifestRes = await axios.get(manifestUrl, { headers: { 'Authorization': token, 'Accept
': 'application/json' } });
    const attachments = manifestRes.data.d.results;

    // Locate the main body file text indicator
    const bodyAttachment = attachments.find(a => a.Name.toLowerCase().includes('body')) || a.Name.
toLowerCase().includes('payload'));
    if (!bodyAttachment) return res.status(400).json({ error: "Trace expired or payload unavailab
le." });

    // Step 2: Stream Data directly into Volatile Server RAM Variable Scope
    const dataUrl = `${CPI_API_URL}/api/v1/MessageProcessingLogAttachments('${bodyAttachment.Id}'
)$value`;
    const payloadStream = await axios.get(dataUrl, { headers: { 'Authorization': token } });
    const dynamicPayloadInRam = payloadStream.data; // Exists purely inside volatile RAM scope

    // Step 3: Immediate Outbound Proxy Route to the target iFlow Sender Endpoint
    const runtimeUrl = `https://your-runtime-worker.itc.cfapps.eu10.hana.ondemand.com${targetIflo
wHttpAddress}`;
    await axios.post(runtimeUrl, dynamicPayloadInRam, { headers: { 'Content-Type': 'application/j
son' } });

    // Step 4: Finalize Context & De-allocate Memory Block
    // Local references fall out of execution scope here, alerting the V8 Garbage Collector to wi
pe the RAM blocks.
    res.status(200).json({ success: true, message: "Message dynamically scrapped, re-queued, and m
```

```
memory flushed." } } ;  
  
    } catch (err) {  
        res.status(500).json({ error: "Dynamic proxy loop execution crash: " + err.message } ) ;  
    }  
    } } ;
```

5. Architectural Concept: Server RAM Pass-Through Lifecycle

By implementing this dynamic framework, data handling follows a strict **In-Memory Execution Pathway** rather than persistent disk storage layouts:

- **Zero Hard-Drive Footprint:** Payloads are never committed to permanent solid-state drives (SSD) or disk cache tables. Data is manipulated purely within the electrical state of high-speed memory chips.
- **Scoping Safety Net:** By maintaining variables strictly inside the short-lived scope of a single HTTP POST request block, the runtime environment cannot preserve the data once the connection context closes.
- **Crash Clean Slate:** If the dashboard server encounters a runtime crash or power cycling event, 100% of the active in-transit records are erased instantaneously, leaving no storage artifacts behind for compliance auditing.

End of Document Architecture Specification - Proprietary System Standard